OPERATING SYSTEM- CSE2104

MODULE 3:

Process Synchronization and Deadlocks

Process Synchronization:

The Critical Section Problem, Peterson's Solution, Synchronization Hardware, Semaphores, Classical Problems of Synchronization

Deadlocks:

System Model, Deadlocks Characterization, Methods for Handling Deadlocks, Deadlock Prevention, Deadlock Avoidance, Deadlock Detection and Recovery from Deadlock

MODULE 3: LECTURE 1 - PROCESS SYNCHRONIZATION, RACE CONDITION, CRITICAL SECTION PROBLEM

PROCESS: TYPES

A process is an instance of a program that is being executed by the operating system

There are basically two types of processes:

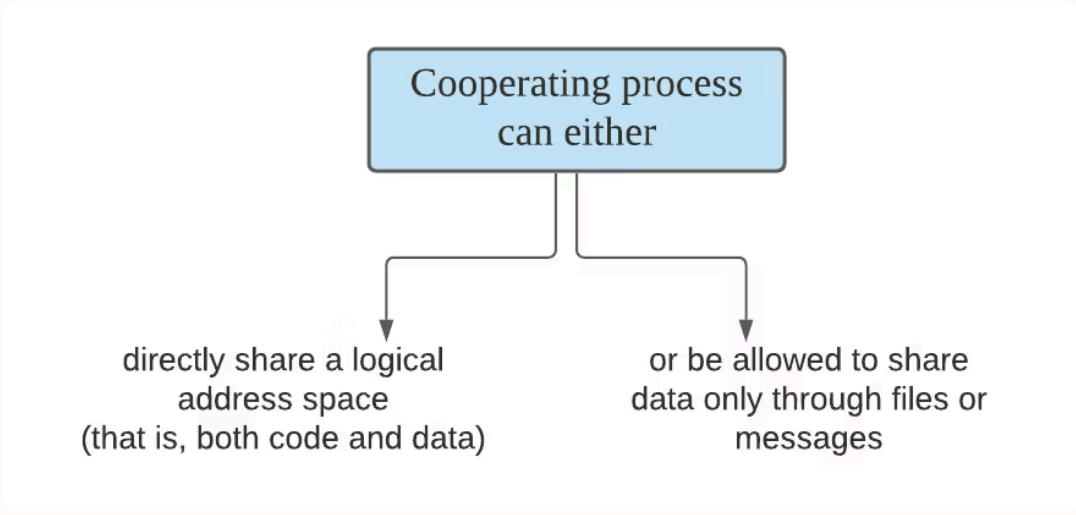
1. Independent Process

Process that runs independently without sharing data (variable, database, files, etc.) with other processes, and its execution neither affects nor is affected by other processes

2. Cooperating Process

Process that can affect or be affected by other processes during execution because they share data (variable, database, files, etc.) or communicate with each other

It **needs inter - process communication (IPC)** to communicate and coordinate its actions



INTER - PROCESS COMMUNICATION (IPC)

Inter - process Communication (IPC) is a mechanism that allows different processes to communicate with each other and coordinate their activities

This communication enables processes to cooperate while executing tasks

Processes can communicate using two main methods:

1

Message Passing Method

Processes exchange information by sending and receiving messages

2

Shared Memory Method

Processes communicate by accessing a common shared memory space

MESSAGE PASSING METHOD

In this method, processes communicate with each other without using any kind of shared memory

If two processes P1 and P2 want to communicate with each other, they proceed as follows:

1. Establish a communication link (if a link already exists, no need to establish it again)
2. Start exchanging messages using basic primitives (at least two):
   - send (message, destination) or send (message)
   - receive (message, host) or receive (message)

SHARED MEMORY METHOD

PRODUCER - CONSUMER PROBLEM

In the shared memory method, two processes communicate with each other using common shared memory

The shared memory is present in the address space of the communicating processes

- **Classic Example: Producer - Consumer Problem**

In an operating system, **producer is a process which is able to produce data/ item and consumer is a process that is able to consume the data/ item produced by the producer**

Both producer and consumer share a common memory buffer

Buffer is a space of a certain size in the memory of the system which is used for storage

The producer produces the data into the buffer and the consumer consumes the data from the buffer

So, what are the Producer-Consumer Problems?

- Producer process should not produce any data when the shared buffer is full
- Consumer process should not consume any data when the shared buffer is empty
- The access to the shared buffer should be mutually exclusive i.e. at a time only one process should be able to access the shared buffer and make changes to it

To maintain consistent data between the producer and consumer processes, proper process synchronization is required to solve the producer–consumer problem

Types of Buffers:

- **Unbounded buffer**
  - It places no practical limit on the size of the buffer
  - Consumer may have to wait for new items but the producer can always produce new items
- **Bounded buffer**
  - Assumes a fixed buffer size
  - Consumer must wait if the buffer is empty, and the producer must wait if the buffer is full

SHARED MEMORY METHOD

PRODUCER - CONSUMER PROBLEM

SHARED MEMORY WITH BOUNDED BUFFER

```
#define BUFFER_SIZE 10

typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

The shared buffer is implemented as a circular array with two logical pointers: **in** and **out**

The variable **in** points to the next free position in the buffer; **out** points to the first full position in the buffer

The buffer is empty when **in == out**; the buffer is full when **((in +1)% BUFFER SIZE) == out**

CODE FOR PRODUCER - CONSUMER PROCESS

PRODUCER

```
while (true) {
     /* produce an item in next_produced */

     while (count == BUFFER_SIZE)
        ; /* do nothing */

     buffer[in] = next_produced;
     in = (in + 1) % BUFFER_SIZE;
     count++;
}
```

CONSUMER

```
while (true) {
     while (count == 0)
        ; /* do nothing */

     next_consumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
     count--;

     /* consume the item in next_consumed */
}
```

Example:

1. **BUFFER SIZE = 10 [_ _ _ _ _ _ _ _ _ _]**
2. **Count = 5**

   i.e. 5 items are currently in the buffer; 5 filled slots and 5 empty slots

```
[A B C D E _ _ _ _ _]
```

Explanation:

Position	Content
0	A
1	B
2	C
3	D
4	E
5	Empty
6	Empty
7	Empty
8	Empty
9	Empty

Producer Adds One Item...

Producer produces F
buffer[5] = F
Buffer becomes:
[A B C D E F _ _ _ _]

Update:
count = count + 1
count = 6

Now:
6 filled slots
4 empty slots

And Then Consumer Removes One Item

Consumer consumes from the start
item = A
Buffer becomes:
[_ B C D E F _ _ _ _]

Update:
count = count - 1
count = 5

Now:
5 filled slots
5 empty slots

SHARED MEMEORY METHOD

PRODUCER - CONSUMER PROBLEM

Producer

```
count++;
```

Consumer

```
count--;
```

count++ steps

1. Read count
2. Increment value
3. Write back to count

Example:

```
Register1 = count
Register1 = Register1 + 1
count = Register1
```

count-- steps

1. Read count
2. Decrement value
3. Write back to count

Example:

```
Register2 = count
Register2 = Register2 - 1
count = Register2
```

Concurrent Execution Example (When producer and consumer process executes at the same time)

Assume:

count = 5

Producer executes **count++** Consumer executes **count--** **Both at the same time**

Initial:

```
Buffer size = 10
Items = 5
```

```
[A B C D E _ _ _ _ _]
count = 5
```

Producer:

```
[A B C D E F _ _ _ _]
count = 6
```

Consumer:

```
[_ B C D E _ _ _ _ _]
count = 4
```

Step	Producer (count++)	Consumer (count--)	count
1	Read count → R1 = 5		5
2		Read count → R2 = 5	5
3	R1 = R1 + 1 → 6		5
4		R2 = R2 - 1 → 4	5
5	Write count = 6		6
6		Write count = 4	4

Producer	Consumer
Executes **count++**	Executes **count--**
Reads 5	Also reads 5
Calculates 5 + 1 = 6	Calculates 5 - 1 = 4
Writes 6 **(wrong result)**	Write 4 **(wrong result)**

This leads to Race Condition (When producer and consumer process executes at the same time)

RACE CONDITION

When more than one process is either executing the same code or accessing the same memory or any shared variable; there is a possibility that the output or value of the shared variable is incorrect, so all the processes race to say that my output is correct

This is commonly referred to as a race condition

PROCESS SYNCHRONIZATION

When multiple processes execute concurrently sharing system resources, inconsistent results might be produced if they try to access or modify the resource simultaneously without proper synchronization

Process synchronization ensures that the multiple processes executing concurrently access shared resources in a controlled and orderly manner, preventing conflicts and avoiding inconsistent or incorrect results

THE CRITICAL SECTION PROBLEM

Concurrent access to shared resources can cause unexpected or misleading outcomes in parallel programming, hence areas of the program where the shared resource is used must be protected in ways that prevent concurrent access

The critical section, often known as the critical region, is considered a protected area in a particular code segment

Critical Section: The term "critical section" refers to a code segment that is accessed by several programs

It includes shared variables or resources that must be synchronized in order to ensure data consistency

Example: Consider a system with n processes (P0, P1, ..., Pn-1). Every process has a critical section of code in which the process may change common variables, update a table, write a file, and so on

When one process is running in its critical section, no other process is permitted to run in that area

There are two methods that control the entry and exit from the critical section:

The **wait()** method controls the entry to the critical section, whereas the **signal()** function controls the exit

Entry Section: It is a step in the process that determines whether or not a process may begin a Critical Section

This section enables a single process to access and modify a shared variable

Exit Section: Other processes waiting in the Entry Section are able to enter the Critical Section through the Exit Section

It also ensures that a process that has completed its execution gets deleted via this section

Remainder Section: The Remainder Section refers to the rest of the code that isn't in the Critical, Entry, or Exit Sections

KEY REQUIREMENTS OF SOLVING CRITICAL SECTION PROBLEM

Mutual Exclusion

- Only one process can execute in its critical section at a time

Progress

- If no process is in its critical section, and some processes want to enter, one of them should be allowed to proceed

Bounded Waiting

- There should be a limit on the number of times other processes can enter their critical sections after a process requests entry and before its request is granted

MODULE 3: LECTURE 2 - PETERSON'S SOLUTION, SYNCHRONIZATION HARDWARE

SOFTWARE BASED SOLUTION FOR CRITICAL SECTION PROBLEM

PETERSON'S ALGORITHM

Peterson's Algorithm is a software-based solution to the critical section problem designed specifically for two processes
It guarantees mutual exclusion and ensures that no process is starved

Key Concepts:

Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections

The processes are numbered Po and P1 or Pi and Pj where j = 1 - i

Peterson's solution requires the two processes to share two **data items**:

int turn;

boolean flag[2];

1. **Flag array**: Two flags; `flag[0]` and `flag[1]`, represent the intention of each process to enter the critical section
2. **Turn** : A shared variable `turn` is used to determine which process gets priority when both processes want to enter the critical section simultaneously

Algorithm (For two processes):

Let i and j represent the process (0 or 1)

1. **Entry Section**:
   - Set `flag[i] = true` to indicate the process intends to enter the critical section
   - Set `turn = j` (where j is the other process) to give priority to the other process
2. **Critical Section**:
   - Once the process gets control, it can safely enter the critical section, knowing that no other process will be in it simultaneously
3. **Exit Section**:
   - Set `flag[i] = false` to indicate the process has finished execution in the critical section
4. **Remainder Section**:
   - This is where the process does other tasks, not related to the critical section

Structure of Process Pi and Pj:

Structure of Process Pi

```
while(true) {
flag[i] = true;    //Pi wants to enter
turn = j;          //Give other process a chance

while (flag[j] == true && turn == j)
    ; // wait (busy waiting)

// Critical Section

   flag[i] = false;     // Pi is leaving

   //  Remainder Section
}
```

Structure of Process Pj

```
while(true) {
flag[j] = true;    //Pj wants to enter
turn = i;          //Give other process a chance

while (flag[i] == true && turn == i)
       ;// wait (busy waiting)

// Critical Section

   flag[j] = false;     // Pj is leaving

   //  Remainder Section
}
```

- **To enter the critical section, process Pi first sets flag [i] to be true and then sets turn to the value j, there by asserting that if the other process wishes to enter the critical section, it can do so**
- **If both processes try to enter at the same time, turn will be set to both i and j at roughly the same time**
  Only one of these assignments will last, the other will occur but will be over written immediately
- **The eventual value of turn determines which of the two processes is allowed to enter its critical section first**

Process	Sets flag	Sets turn	Wait condition
Pi	flag [i]	turn = j	flag [j] && turn == j
Pj	flag [j]	turn = i	flag [i] && turn == i

To prove that solution is correct, then we need to show that

1 Mutual exclusion is preserved

Both processes cannot enter together because of `turn`

2 Progress requirement is satisfied

If one process is not interested (`flag = false`), the other enters immediately

3 Bounded-waiting requirement is met

`turn` ensures **alternate chances**

Prevents one process from dominating

SOFTWARE BASED SOLUTION FOR CRITICAL SECTION PROBLEM

PETERSON'S SOLUTION

1. To Prove Mutual Exclusion

- Each Pi enters its critical section only if either flag [j] == false or turn == i
- If we assume that both the processes are executing in their critical sections at the same time, then flag [0] == flag [1] == true and turn == i == j
- These two observations imply that Pi and Pj could not have successfully executed their while statements at the same time, since the value of turn can be either 0 or 1 but cannot be both
- Hence, one of the processes (Pj) must have successfully executed the while statement, whereas Pi had to execute at least one additional statement ("turn == j")
- However, at that time, flag [j] == true and turn == j, and this condition will persist as long as Pi is in its critical section, as a result, mutual exclusion is preserved

2. To Prove Progress and Bounded-waiting

- A process Pi can be prevented from entering the critical section only if it is stuck in the while loop with the condition flag [j] == true and turn == j
- If Pj is not ready to enter the critical section, then flag [j] == false, and Pi can enter its critical section
- If Pj has set flag [j] = true and is also executing in its while statement, then either turn == i or turn == j
  - If turn == i, then Pi will enter the critical section
  - If turn == j, then Pj will enter the critical section
- However, once Pj exits its critical section, it will reset flag [j] = false, allowing Pi to enter its critical section
- If Pj resets flag [j] to true, it must also set turn to i
- Thus, since Pi does not change the value of the variable turn while executing the while statement, Pi will enter the critical section (progress) after at most one entry by Pj (bounded waiting)

HARDWARE BASED SOLUTION FOR CRITICAL SECTION PROBLEM

SYNCHRONIZATION HARDWARE

A hardware solution for the critical section problem

There is a shared lock variable which can take either of the two values, 0 (unlocked) or 1 (locked)

Before entering into critical section, a process inquires about the lock

If it is locked, it keeps on waiting till it becomes free

If it is not locked, it takes the lock and executes the critical section

TEST-AND-SET LOCK

Test-and-Set Lock is a hardware synchronization mechanism that uses an atomic operation to control access to the critical section

- It uses a shared variable called **lock**
- Only one process can hold the lock at a time

```
do {
    acquire lock
            critical section
    release lock
            remainder section
} while (TRUE);
```

Definition of TestAndSet () Instruction

```
boolean TestAndSet (boolean *target)
{
    booleanrv = *target;
    *target = TRUE;
    return rv:
}
```

Figure: The definition of the TestAndSet () instruction.

- target is a **pointer to a shared variable** (usually called lock)
- Read the current value of target
- Store it in rv (return value) (if lock = 0 then rv= 0)
- *target = TRUE; Set the lock to **TRUE (locked)**
- return rv; Return the **old value** of lock

This is an atomic operation (runs in one step)

Implementation of Test and Set Lock

Process P1

```
do {
    while (test_and_set(&lock))
        ; /* do nothing */
            /* critical section */
    lock = false;
            /* remainder section */
 } while (true);
```

Process P2

```
do {
    while (test_and_set(&lock))
        ; /* do nothing */
            /* critical section */
    lock = false;
            /* remainder section */
 } while (true);
```

Note: Initial value of lock = 0
while (0): False
while (1): True

Step by Step Execution Steps For Two Processes (P1 and P2)

Step	Lock Value	P1 Action	P2 Action	Explanation
1	0	test_and_set → returns 0 → enters CS	—	Lock was free; P1 enters
2	1	In Critical Section	test_and_set → returns 1	P1 is executing CS; P2 sees lock busy
3	1	Still inside	Keeps checking (loop)	P2 is **busy waiting**
4	1 → 0	P1 exits, sets lock = 0	Waiting	Lock becomes free
5	0	—	test_and_set → returns 0 → enters CS	P2 enters CS
6	1	—	In Critical Section	P2 is executing CS
7	1 → 0	—	P2 exits, sets lock = 0	Lock free again

1

This code represents a solution to the critical section problem using the test-and-set lock mechanism

2

In this approach, each process continuously attempts to enter its critical section by executing the
while (test_and_set(&lock)) loop

3

The **test_and_set** operation is atomic; meaning it Reads + Modifies + Writes the value of the shared variable **lock** as a one indivisible action (without interruption)
Only hardware can guarantee it

4

If the lock was previously false, the function returns false, allowing the process to exit the loop and enter the critical section

5

If the lock is already true, it means another process is in the critical section, so the loop continues and the process waits (busy waiting)

6

Once inside the critical section, the process performs its task and then sets
lock = false to release the lock

7

This allows other waiting processes to acquire the lock and enter the critical section
After releasing the lock, the process executes its remainder section and repeats the cycle indefinitely

This solution ensures mutual exclusion, as only one process can hold the lock at a time, but it suffers from drawbacks such as busy waiting and the possibility of starvation due to the lack of bounded waiting

MODULE 3: LECTURE 3 - SEMAPHORES, CLASSICAL PROBLEM OF SYNCHRONIZATION

SOFTWARE BASED SOLUTION FOR CRITICAL SECTION PROBLEM

SEMAPHORE

A semaphore is a synchronization tool in operating systems, specifically an integer variable used to manage concurrent access to shared resources by multiple processes

Semaphore S is an integer variable accessed only through two standard atomic operations:
wait ( ) (P operation) [used to test or decrease value]
signal ( ) (V operation) [used to increment]

Definition of wait ()

```
wait (S) {
        while S <= 0
        ; // no-op
    S--;
```

Definition of signal ()

```
signal (S) {
        S++;}
```

All the modifications to the integer value of the semaphore S in wait () and signal () operations must be executed indivisibly
That is, when one process modifies the semaphore value, no other process can simultaneously modify that semaphore value

Types of Semaphores

1. Binary Semaphore (0 or 1)

- Can take only two values: 0 or 1
- Also known as mutex locks, as they are the locks that provide mutual exclusion
  - mutex = 1 → Resource is free (Unlocked)
  - mutex = 0 → Resource is busy (Locked)

- **How It Works**
  - Initial Value

```
mutex = 1
```

  - Process Execution

```
do {
wait(mutex);   // mutex = 0 → enter
    // critical section
signal(mutex); // mutex = 1 → leave
    // Remainder section
} while (true)
```

    - Example
      - Process wants to enter critical section

```
wait(mutex);
```

        - If mutex = 1 → becomes 0 → process enters
        - If mutex = 0 → process waits (result is negative so process waits)

      - After Finishing

```
signal(mutex);
```

        - Value becomes 1 → next process can enter

2. Counting Semaphore

- Can take any non-negative integer value (0, 1, 2, 3, ...)
- Represents number of available resources

- **How It Works**
  - Initial Value

```
mutex = 3  (3 resources available)
```

  - Process Execution

```
wait(S);   // S-- (take resource)
// critical section
signal(S); // S++ (release resource)
```

    - Example

      3 printers available:

      - P0 → S = 2 → uses printer
      - P1 → S = 1 → uses printer
      - P2 → S = 0 → uses printer
      - P3 → S = 0 → must wait

      When one finishes:

      - signal(S) → S = 1 → P3 can enter

Semaphores provide a mechanism to enforce mutual exclusion and prevent race conditions, ensuring that only one process/ thread can access a shared resource at a time

Features of Semaphores

- **Mutual Exclusion**: Semaphore ensures that only one process accesses a shared resource at a time
- **Process Synchronization**: Semaphore coordinates the execution order of multiple processes
- **Resource Management**: Limits access to a finite set of resources, like printers, devices, etc.
- **Reader-Writer Problem**: Allows multiple readers but restricts the writers until no reader is present
- **Avoiding Deadlocks**: Prevents deadlocks by controlling the order of allocation of resources

CLASSICAL PROBLEMS OF SYNCHRONIZATION

1. BOUNDED - BUFFER PROBLEM
2. DINING - PHILOSOPHER PROBLEM
3. READER - WRITER PROBLEM

CLASSICAL PROBLEMS OF SYNCHRONIZATION

BOUNDED - BUFFER PROBLEM

The bounded - buffer problem, which is also called the producer - consumer problem, is one of the classic problems of synchronization

Solution to Producer - Consumer Problem

One solution to Producer - Consumer problem is to use semaphores

Three semaphores will be used here:

1. `m`, a binary semaphore that is used to acquire and release the lock
2. `empty`, a counting semaphore whose initial value is the number of slots in the buffer, since, initially all slots are empty
3. `full`, a counting semaphore whose initial value is `0`

At any instant, the current value of empty represents the number of empty slots in the buffer and full represents the number of occupied slots in the buffer

Producer Operation

Pseudocode of the producer function looks like:

```
do
{
    // wait until empty > 0 and then decrement 'empty'
    wait(empty);
    // acquire lock
    wait(mutex);

    /* perform the insert operation in a slot */

    // release lock
    signal(mutex);
    // increment 'full'
    signal(full);
}
while(TRUE)
```

1. The producer first waits until there is at least one empty slot
2. It decrements the empty semaphore because there will now be one less empty slot since the producer is going to insert data in one of those slots
3. It acquires a lock on the buffer, so that the consumer cannot access the buffer until the producer completes its operation
4. After performing the insert operation, the lock is released and the value of full is incremented because the producer has just filled a slot in the buffer

Consumer Operation

Pseudocode for the consumer function looks like:

```
do
{
    // wait until full > 0 and then decrement 'full'
    wait(full);
    // acquire the lock
    wait(mutex);

    /* perform the remove operation in a slot */

    // release the lock
    signal(mutex);
    // increment 'empty'
    signal(empty);
}
while(TRUE);
```

1. The consumer waits until there is at least one full slot in the buffer
2. It decrements the full semaphore because the number of occupied slots will be decreased by one after the consumer completes its operation
3. It acquires a lock on the buffer, so that the producer cannot access the buffer until the consumer completes its operation
4. After performing the removal operation, the lock is released and the empty semaphore is incremented by 1, because the consumer has just removed data from an occupied slot

CLASSICAL PROBLEMS OF SYNCHRONIZATION

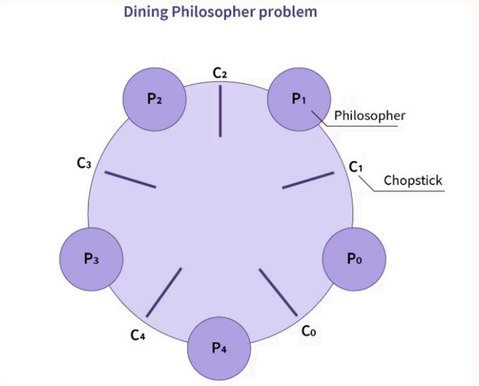
DINING - PHILOSOPHER PROBLEM

The dining philosophers problem is another classic synchronization problem

It is used to evaluate the situations where there is a need of allocating multiple resources to multiple processes

Problem Statement

Consider there are five philosophers sitting around a circular dining table which has five chopsticks and a bowl of rice in the middle



- At any instant, a philosopher is either eating or thinking
- When a philosopher wants to eat, he uses two chopsticks — one from his left and one from his right
- A philosopher is allowed to pick only one chopstick at a time
- When a philosopher wants to think, he keeps down both chopsticks at their original place

Solution

From the problem statement, it is clear that a philosopher is in an endless cycle of thinking and eating

A philosopher can think for an indefinite amount of time; but when a philosopher starts eating, he has to stop at some point in time

An array of five semaphores, `stick[5]` will be used here for each of the five chopsticks

Pseudocode for each philosopher looks like:

```
while(TRUE)
{
    wait(stick[i]);
    /*
        mod is used because if i=5, next
        chopstick is 1 (dining table is ciroular)
    */
    wait(stick[(i+1) % 5]);

    /* eat */
    signal(stick[i]);

    signal(stick[(i+1) % 5]);
    /* think */
}
```

1. When a philosopher wants to eat the rice, he will wait for the chopstick at his left and picks up that chopstick
2. Then he waits for the right chopstick to be available, and then picks it too
3. After eating, he puts both the chopsticks down

If all the five philosophers are hungry simultaneously, and each of them picks up one chopstick, then a deadlock situation occurs because they will be waiting for another chopstick forever

The possible solutions for this situation are:

1. There should be at most four philosophers on the table while maintaining the number of chopsticks to five
2. An even philosopher should pick the right chopstick and then the left chopstick while an odd philosopher should pick the left chopstick and then the right chopstick
3. A philosopher should only be allowed to pick their chopstick if both are available at the same time

CLASSICAL PROBLEMS OF SYNCHRONIZATION

READER - WRITER PROBLEM

The Reader-Writer Problem is a classical OS synchronization problem
It refers to managing access of multiple processes to shared data (like a file or a database)

Reader

A process that only retrieves (reads) data from a shared data (like a file or a database) without modifying it

Writer

A process that modifies (updates or writes) the shared data (like a file or a database)

Problem Statement

To allow multiple readers to read simultaneously while ensuring only one writer can modify data at a time, preventing data corruption

1. Multiple Readers can access the resource at the same time
2. Only one Writer can modify the data at a time
3. No Reader can read while a Writer is actively writing
4. No Writer can write while a Reader is currently reading

Solution

- In this problem, we will assume that the readers have higher priority than the writers
- Hence, if a writer wants to write to the resource, it must wait until there are no readers currently accessing that resource

Two semaphores and an integer variable will be used here:

1. m, a semaphore (initialized to 1) which is used to ensure mutual exclusion when read_count is updated i.e. when any reader enters or exit from the critical section
2. w, a semaphore (initialized to 1) common to both reader and writer processes
3. read_count, an integer variable (initialized to 0) that keeps track of how many processes are currently reading the object

Code for the writer process looks like:

```
while(TRUE)
{
    wait(w);

   /* perform the write operation */

   signal(w);
}
```

1. Writer just waits on the w semaphore until it gets a chance to write to the resource
2. After performing the write operation, it increments w so that the next writer can access the resource

Code for the reader process looks like:

```
while(TRUE)
{
    //acquire lock
    wait(m);
    read_count++;
    if(read_count == 1)
        wait(w);

    //release lock
    signal(m);

    /* perform the reading operation */
    }
    // acquire lock
    wait(m);
    read_count--;
    if(read_count == 0)
        signal(w);
    }
    // release lock
    signal(m);
}
```

1. A reader process acquires a lock whenever it updates the read_count
2. When a reader wants to access the resource, it increments the read_count value, accesses the resource, and then decrements the read_count value
3. Semaphore w is used by the first reader which enters the critical section and the last reader who exits the critical section
4. The reason for this is, when the first reader enters the critical section, the writer is blocked from the resource and hence, only new readers can access the resource now
5. Similarly, when the last reader exits the critical section, it signals the writer using the w semaphore because there are zero readers now and a writer can have the chance to access the resource

Possibilities

Case	Process 1	Process 2	Allowed / Not Allowed
Case 1	Writing	Writing	Not Allowed
Case 2	Reading	Writing	Not Allowed
Case 3	Writing	Reading	Not Allowed
Case 4	Reading	Reading	Allowed

MODULE 3: LECTURE 4 - DEADLOCK, DEADLOCKS CHARACTERIZATION

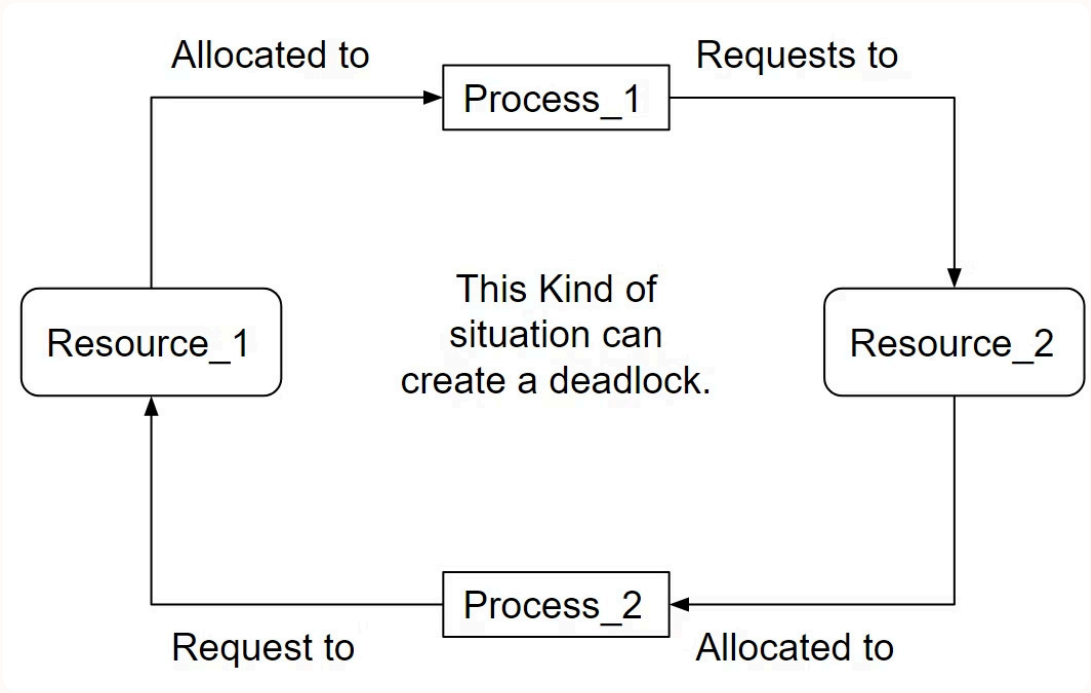
WHAT IS DEADLOCK?

- Deadlock in OS refers to a situation where a group of processes or threads is not able to proceed because each of the processes or threads is waiting for the other to release a resource

- Deadlock usually occurs in systems where multiple processes compete for limited resources such as CPU time, memory or input/ output devices

WHAT IS DEADLOCK?

EXAMPLE



Two processes, Process_1 and Process_2 are holding a resource

Process_1 is holding the Resource_1 while waiting for Resource_2

Process_2 has been allocated with the Resource_2 and is waiting for Resource_1

Both the processes are waiting for each other to release the resource (Process_1 awaits the release of Resource_2 whereas Process_2 waits for Resource_1 to get released) which never happens

This is referred to as a deadlock

NECESSARY CONDITIONS FOR DEADLOCK

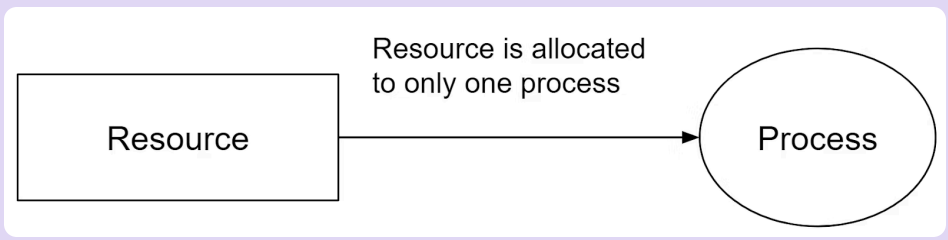
There are four necessary conditions for a Deadlock to occur, often referred to as Deadlock Conditions

1

Mutual Exclusion

At least one resource must be held in a non-shareable mode, meaning only one process can use it at any given time

This condition ensures that a resource cannot be simultaneously accessed or modified by multiple processes



2

Circular Wait

A closed chain of processes exists, where each process holds a resource needed by the next process in the chain, creating a circular dependency

Example:

A set of processes $\{P0, P1, P2, P3\}$ are waiting for each other in a circular fashion

such that $P0$ depends on $P1$, $P1$ depends on $P2$, $P2$ depends on $P3$ and $P3$ depends on $P0$

This creates a circular relation between all these processes and they have to wait forever to be executed

3

Hold and Wait

A process is currently holding at least one resource while waiting to acquire additional resources that are held by other processes

This condition can lead to a situation where processes hold some resources and wait indefinitely for others

4

No Pre-emption

Resources cannot be pre-empted or forcibly taken away from a process; they must be released voluntarily by the process holding them

If a process is holding a resource and cannot complete its task due to waiting for another resource, it will not release its current resource, contributing to a potential Deadlock

For a deadlock to be broken, at least one of these conditions must not be met

RESOURCE ALLOCATION GRAPH (RAG)

The **Resource Allocation Graph** in OS is a visual representation that depicts the relationships between **Processes** and **Resources**

RAG gives complete information about the state of a system such as

- How many processes exist in the system?
- How many instances of each resource type exist?
- How many instances of each resource type are allocated?
- How many instances of each resource type are still available?
- How many instances of each resource type are held by each process?
- How many instances of each resource type does each process need for execution?

Components of RAG

There are two major components of a Resource Allocation Graph

1

Vertices

> Process Vertices
> Resource Vertices

Process Vertices

Process vertices represent the processes

They are drawn as a circle by mentioning the name of process inside the circle

Resource Vertices

Resource vertices represent the resources

Depending on the number of instances that exists in the system, resource vertices may be single instance or multiple instance

They are drawn as a rectangle by mentioning the dots inside the rectangle

The number of dots inside the rectangle indicates the **number of instances** of that resource existing in the system

2

Edges

> Assign Edges
> Request Edges

Assign Edges

Assign edges represent the assignment of resources to the processes

They are drawn as an arrow where the head of the arrow points to the process and tail of the process points to the instance of the resource

Request Edges

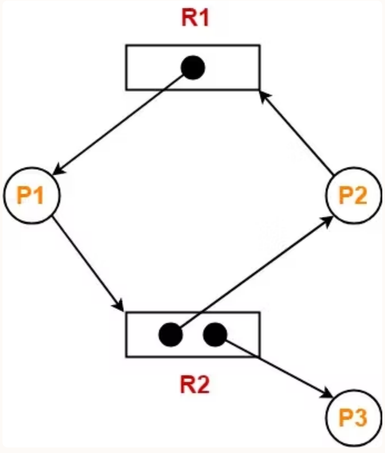
Request edges represent the waiting state of processes for the resources

They are drawn as an arrow where the head of the arrow points to the instance of the resource and tail of the process points to the process

If a process requires 'n' instances of a resource type, then 'n' assign edges will be drawn

Example of RAG

The following diagram represents a Resource Allocation Graph



It gives the following information

- ☐ There exist three processes in the system namely P1, P2 and P3
- ☐ There exist two resources in the system namely R1 and R2
- ☐ There exists a single instance of resource R1 and two instances of resource R2
- ☐ Process P1 holds one instance of resource R1 and is waiting for an instance of resource R2
- ☐ Process P2 holds one instance of resource R2 and is waiting for an instance of resource R1
- ☐ Process P3 holds one instance of resource R2 and is not waiting for anything

MODULE 3: LECTURE 5 - METHODS FOR HANDLING DEADLOCKS: DEADLOCK PREVENTION, DEADLOCK AVOIDANCE, DEADLOCK DETECTION AND RECOVERY FROM DEADLOCK

DEADLOCK HANDLING STRATEGIES

1. Deadlock Prevention
2. Deadlock Avoidance
3. Deadlock Ignorance
4. Deadlock Detection and Recovery

DEADLOCK PREVENTION

Deadlock Prevention strategy involves designing a system that violates one of the four necessary conditions required for the occurrence of deadlock ensuring the system remains free from the deadlock

The various conditions of deadlock occurrence may be violated as

1

Mutual Exclusion

- To violate this condition, all the system resources must be such that they can be used in a shareable mode
- In a system, there are always some resources which are mutually exclusive by nature
- So, this condition can not be violated

2

Hold and Wait

This condition can be violated in the following ways

Approach 01

- In this approach, a process has to first request for all the resources it requires for execution
- Once it has acquired all the resources, only then it can start its execution. This approach ensures that the process does not hold some resources and wait for other resources.
- This approach is less efficient and is not implementable since it is not possible to predict in advance which resources will be required during the execution

Approach 02

- In this approach, a process is allowed to acquire the resources it desires at the current moment
- After acquiring the resources, it start its execution
- Now before making any new request, it has to compulsorily release all the resources that it holds currently
- This approach is efficient and implementable

Approach 03

- In this approach, a timer is set after the process acquires any resource
- After the timer expires, a process has to compulsorily release the resource

3

No Pre-emption

- This condition can by violated by forceful pre-emption
- Consider a process is holding some resources and request other resources that can not be immediately allocated to it
- Then, by forcefully pre-empting the currently held resources, the condition can be violated
- A process is allowed to forcefully pre-empt the resources possessed by some other process only if it is a high priority process or a system process
- The victim process is in the waiting state

4

Circular Wait

- This condition can be violated by not allowing the processes to wait for resources in a cyclic manner
- To violate this condition, the following approach is followed:
  - A natural number is assigned to every resource
  - Each process is allowed to request for the resources either in only increasing or only decreasing order of the resource number
  - In case increasing order is followed, if a process requires a lesser number resource, then it must release all the resources having larger number and vice versa
- This approach is the most practical approach and implementable
- However, this approach may cause starvation but will never lead to deadlock

DEADLOCK AVOIDANCE

Deadlock Avoidance strategy involves maintaining a set of data using which a decision is made whether to entertain the new request or not

If entertaining the new request causes the system to move in an unsafe state, then it is discarded

This strategy requires that every process declares its maximum requirement of each resource type in the beginning

The main challenge with this approach is predicting the requirement of the processes before execution

Banker's Algorithm is an example of a deadlock avoidance strategy

BANKER'S ALGORITHM

Banker's Algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems

It ensures the existence of at least one sequence of processes such that each process can obtain the needed resources to complete its execution

It helps prevent situations where programs get stuck and can not finish their tasks

It keeps track of resources each program needs and what's available

IMPLEMENTATION OF BANKER'S ALGORITHM

The following Data structures are used to implement the Banker's Algorithm:

Let 'n' be the number of processes in the system and 'm' be the number of resource types.

1 Available

It is a 1-D array of size 'm' indicating the number of available resources of each type.

Available[j] = k means there are 'k' instances of resource type Rj

2 Max

It is a 2-d array of size 'n*m' that defines the maximum demand of each process in a system.

Max[i, j] = k means process Pi may request at most 'k' instances of resource type Rj.

3 Allocation

It is a 2-d array of size 'n*m' that defines the number of resources of each type currently allocated to each process.

Allocation[i, j] = k means process Pi is currently allocated 'k' instances of resource type Rj

4 Need

It is a 2-d array of size 'n*m' that indicates the remaining resource need of each process.

Need [i, j] = k means process Pi currently needs 'k' instances of resource type Rj

Need [i, j] = Max [i, j] – Allocation [i, j]

Allocation specifies the resources currently allocated to process Pi and

Need specifies the additional resources that process Pi may still request to complete its task.

Key Concepts in Banker's Algorithm

1

Safe State:

There exists at least one sequence of processes such that each process can obtain the needed resources, complete its execution, release its resources, and thus allow other processes to eventually complete without entering a deadlock.

2

Unsafe State:

Even though the system can still allocate resources to some processes, there is no guarantee that all processes can finish without potentially causing a deadlock.

EXAMPLE OF UNSAFE STATE

Consider a system with three processes (P1, P2, P3) and 7 instances of a resource. Let's say:

- [ ] **The available instances of the resource are: 1**
- [ ] **The current allocation of resources to processes is:**
  - **P1: 2 P2: 3 P3: 1**
- [ ] The maximum demand (maximum resources each process may eventually request) is:
  - **P1: 4 P2: 5 P3: 3**

In this situation:

> ***Need = Maximum Demand - Allocation***
>
> ***To Check Safe State*** ***Need <=*** **Available**

Process	Maximum Demand	Allocation	Need
P1	4	2	2
P2	5	3	2
P3	3	1	2

- `P1` **may need 2 more resources to complete.**
- `P2` **may need 2 more resource to complete.**
- `P3` **may need 2 more resources to complete.**

However, there is only 1 resource available. Even though none of the processes are currently deadlocked, the system is in an unsafe state because there is no sequence of resource allocation that guarantees all processes can complete.

COMPONENTS OF BANKER'S ALGORITHM

The Banker’s Algorithm is a deadlock avoidance algorithm that works using two components

1

Safety Algorithm

Checks whether the system is in a safe state

(Meaning all processes can complete without causing deadlock)

1. Initialize:

Work = Available (resources currently available)

Finish[i] = false for all processes (none have finished yet)

2. Look for a process Pi such that:

Finish[i] = false

Need[i] ≤ Work (resources required can be satisfied)

3. If such a process is found:

Pretend to allocate resources to it: Work = Work + Allocation[i]

Mark the process finished: Finish[i] = true

Repeat step 2 for remaining processes

4. If all processes are marked finished:

(Finish[i] = true for all), the system is safe.

Essentially, the algorithm simulates process completion to verify that all processes can finish safely

2

Resource Request Algorithm

Decides if a process’s resource request can be granted safely

1. Check if the request exceeds the process’s maximum need:

If Request[i] ≤ Need[i], continue; otherwise, error.

2. Check if resources are available:

If Request[i] ≤ Available, continue; otherwise, the process waits.

3. Temporarily allocate the resources:

Available = Available - Request[i]

Allocation[i] = Allocation[i] + Request[i]

Need[i] = Need[i] - Request[i]

4. Run the Safety Algorithm:

If the new state is safe, grant the request.

If unsafe, roll back and make the process wait.

In short, the resource-request algorithm ensures resources are only granted if the system remains safe, preventing deadlock

EXAMPLE

Considering a system with five processes P0 through P4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t0 following snapshot of the system has been taken.

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

▼ Safety Algorithm:

Step 1: Given Data

Total instances:

- A = 10
- B = 5
- C = 7

Available (given):

Available = (3, 3, 2)

Step 2: Allocation and Max Table

Process	Allocation (A B C)	Max (A B C)
P0	0 1 0	7 5 3
P1	2 0 0	3 2 2
P2	3 0 2	9 0 2
P3	2 1 1	2 2 2
P4	0 0 2	4 3 3

Step 3: Calculate Need Matrix

Formula:
Need = Max – Allocation

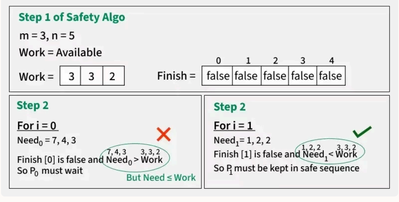
Process	Need (A B C)
P0	(7-0, 5-1, 3-0) = **7 4 3**
P1	(3-2, 2-0, 2-0) = **1 2 2**
P2	(9-3, 0-0, 2-2) = **6 0 0**
P3	(2-2, 2-1, 2-1) = **0 1 1**
P4	(4-0, 3-0, 3-2) = **4 3 1**

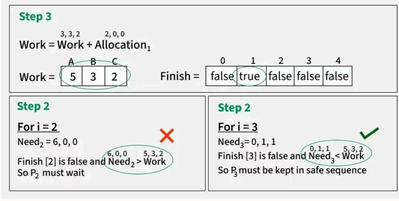
Step 4: Apply Banker's Algorithm

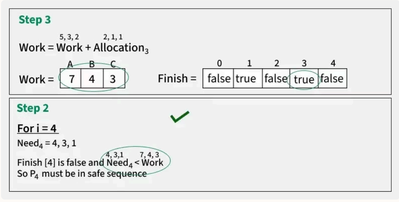
Start with:
Available = (3, 3, 2)

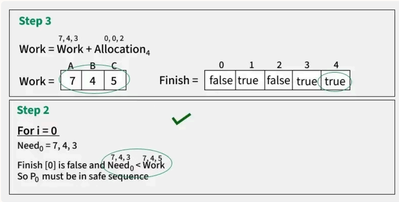
We check which process can execute:
Condition: **Need ≤ Available**

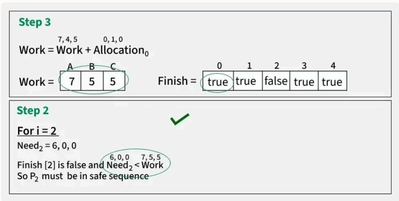
check Finish[i]=true or false (if true- process executes, if false- process will wait)

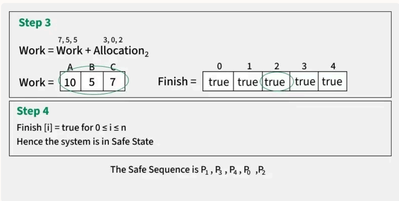












The Safe Sequence is P_1 , P_3 , P_4 , P_0 , P_2

EXAMPLE

Considering a system with five processes P0 through P4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t0 following snapshot of the system has been taken.

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Resource Request Algorithm

Suppose:

Process **P1 requests (1,0,2)**

Step 1: Check if request ≤ Need

Need of P1 = (1,2,2)

Request = (1,0,2)

(1,0,2) ≤ (1,2,2) → OK

Step 2: Check if request ≤ Available

Available = (3,3,2)

(1,0,2) ≤ (3,3,2) → OK

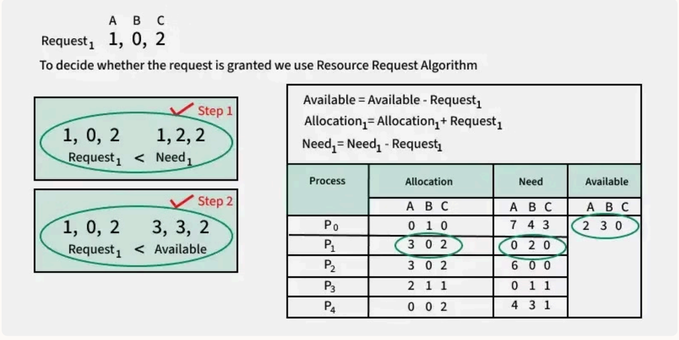
Step 3: Pretend Allocation (Trial)

Available = Available – Request[i]

Allocation[i] = Allocation[i] + Request[i]

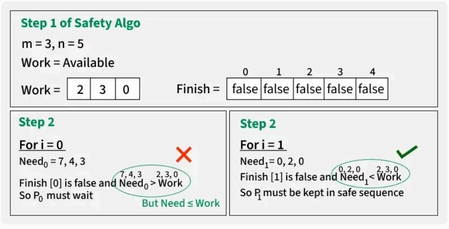
Need[i] = Need[i] – Request[i]

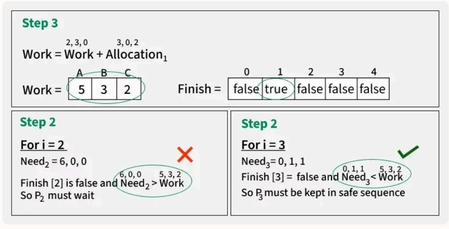
- Available = (3,3,2) – (1,0,2) = **(2,3,0)**
- Allocation(P1) = (2,0,0) + (1,0,2) = **(3,0,2)**
- Need(P1) = (1,2,2) – (1,0,2) = **(0,2,0)**

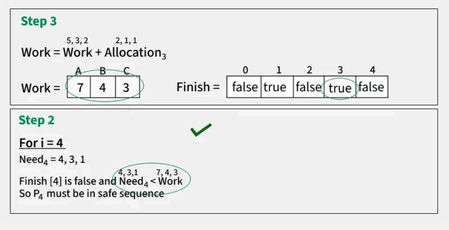


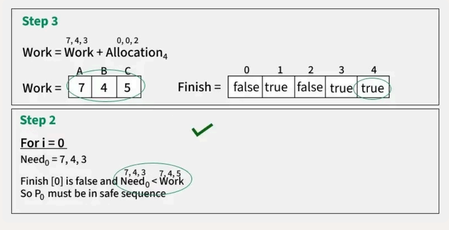
Step 4: Apply Safety Algorithm

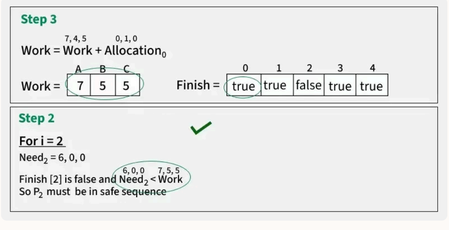
Run Banker's Algorithm again

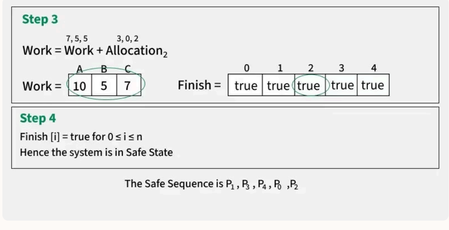












DEADLOCK IGNORANCE

The Deadlock Ignorance strategy simply assumes:

That deadlocks are so rare that it is not worth the cost of preventing or detecting them.

If a deadlock does occur, the operating system may take drastic measures such as rebooting to recover.

This approach is called the Ostrich Algorithm, because it resembles the ostrich burying its head in the sand and pretending the problem doesn't exist

Why Use Deadlock Ignorance?

1. Rarity of Deadlocks:
Deadlocks occur very rarely in well-designed programs. Other failures (hardware crashes, software bugs, compiler errors) are much more common. Engineers prefer to spend effort on frequent problems rather than rare ones.

2. High Overhead of Handling:
Deadlock prevention/ avoidance requires keeping track of resource allocation graphs, running detection cycles or restricting process behavior. These add significant time and space overhead. For most users, ignoring deadlock makes the OS faster and simpler.

3. Practical Philosophy:
For personal computers or single-user systems, rebooting after a rare deadlock is acceptable. For mission-critical systems (e.g., banking, aviation, medical devices), this strategy is not suitable.

Note: many widely used systems, including UNIX and Windows, often adopt deadlock ignorance.

DEADLOCK DETECTION AND RECOVERY

Deadlock detection is the process of identifying when processes are stuck waiting for resources held by other processes.

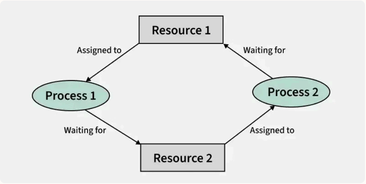
Recovery is the method of resolving the deadlock to allow the system to continue functioning.

Detection is done using techniques like Resource Allocation Graphs (RAG) or Wait-for Graphs.

Once a deadlock is detected, recovery methods include process termination, resource preemption, or process rollback.

Deadlock Detection

1. If Resources Have a Single Instance



In this case for Deadlock detection, we can run an algorithm to check for the cycle in the Resource Allocation Graph. The presence of a cycle in the graph is a sufficient condition for deadlock.

In the diagram, **resource 1** and **resource 2** have single instances. There is a cycle R1 → P1 → R2 → P2. So, Deadlock is Confirmed.

2. If There are Multiple Instances of Resources

Detection of the cycle is necessary but not a sufficient condition for deadlock detection, in this case, the system may or may not be in deadlock varies according to different situations.

For systems with multiple instances of resources, algorithms like Banker's Algorithm can be adapted to periodically check for deadlocks.

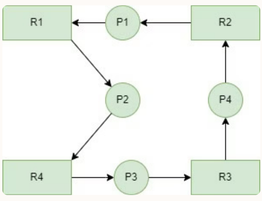
3. Wait-For Graph Algorithm

The Wait-For Graph Algorithm is a deadlock detection algorithm used to detect deadlocks in a system where resources can have multiple instances. The algorithm works by constructing a Wait-For Graph, which is a directed graph that represents the dependencies between processes and resources.

Wait -For Graph Contains only Processes after removing the Resources while conversion from Resource Allocation Graph.

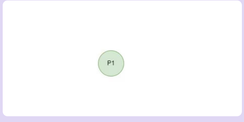
EXAMPLE

Example : Consider a Resource Allocation Graph with 4 Processes P1, P2, P3, P4, and 4 Resources R1, R2, R3, R4. Find if there is a deadlock in the Graph using the Wait for Graph-based deadlock detection algorithm.



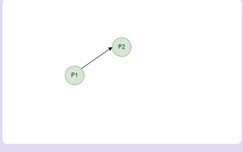
Step 1:

First take Process P1 which is waiting for Resource R1, resource R1 is acquired by Process P2, Start a Wait-for-Graph for the above Resource Allocation Graph.



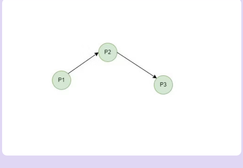
Step 2

Now we can observe that there is a path from P1 to P2 as P1 is waiting for R1 which is been acquired by P2. Now the Graph would be after removing resource R1 looks like



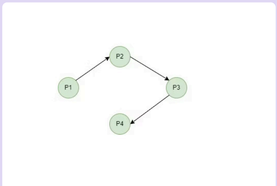
Step 3

From P2 we can observe a path from P2 to P3 as P2 is waiting for R4 which is acquired by P3. So make a path from P2 to P3 after removing resource R4 looks like.



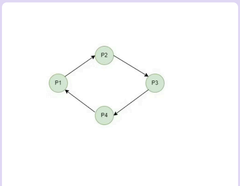
Step 4

From P3 we find a path to P4 as it is waiting for P3 which is acquired by P4. After removing R3 the graph looks like this.



Step 5

Here we can find Process P4 is waiting for R2 which is acquired by P1. So finally the Wait-for-Graph is as follows:



Step 6

Note: Finally In this Graph, we found a cycle as the Process P4 again came back to the Process P1 which is the starting point (i.e., it's a closed-loop). So, According to the Algorithm if we found a closed loop, then the system is in a deadlock state. So here we can say the system is in a deadlock state.

DEADLOCK RECOVERY

Killing The Process:

Killing all the processes involved in the deadlock. Killing process one by one. After killing each process check for deadlock again and keep repeating the process till the system recovers from deadlock. Killing all the processes one by one helps a system to break circular wait conditions.

Process Rollback:

Rollback deadlocked processes to a previously saved state where the deadlock condition did not exist. It requires checkpointing to periodically save the state of processes.

Resource Preemption:

Resources are preempted from the processes involved in the deadlock, and preempted resources are allocated to other processes so that there is a possibility of recovering the system from the deadlock. In this case, the system goes into starvation.

Concurrency Control:

Concurrency control mechanisms prevent data inconsistencies in systems with multiple concurrent processes. They ensure that processes do not access the same data simultaneously, avoiding errors and potential deadlocks. By managing access to shared resources, these mechanisms help maintain system stability and prevent processes from blocking each other.